

ESPECIFICACIÓN TÉCNICA DE DESARROLLO

SISTEMA DE MAPEO INTERACTIVO DE RED CON INTEGRACIÓN DE API PRTG (DJANGO + FRONTEND)

1. OBJETIVO DEL PROYECTO

Desarrollar un módulo web interactivo dentro de una aplicación **Django** que permita clonar, visualizar y manipular libremente (mediante drag-and-drop) la topología de la red de infraestructura. El sistema debe consumir los datos de sensores y dispositivos en tiempo real desde la instancia activa de **Paessler PRTG** alojada en <https://nova.slability.com/>, y mapear dinámicamente sus estados operacionales a un código de colores (Verde/Rojo/Amarillo) sobre el canvas interactivo.

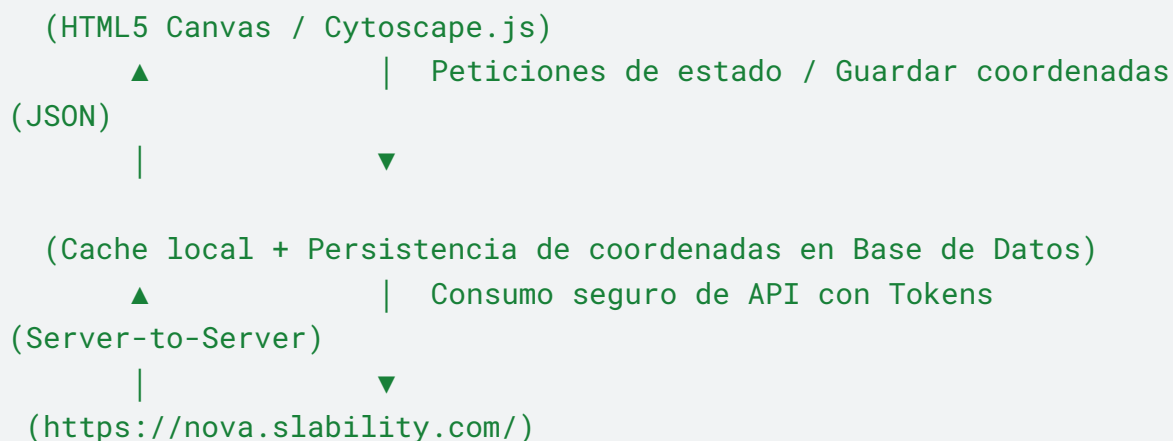
AA

2. ARQUITECTURA GENERAL DEL SISTEMA

El sistema operará bajo un modelo de tres capas para garantizar seguridad, rendimiento y mitigar problemas de CORS (Cross-Origin Resource Sharing) en el navegador:



None



1.

Capa de Datos de Monitoreo (PRTG Core Server): El motor local de PRTG expone la API HTTP REST en <https://nova.slability.com/api/>.

2. **Capa Backend (Django):** Django actúa como middleware de seguridad y proxy. Almacena las coordenadas de visualización de cada nodo de red (que PRTG no guarda de forma nativa de manera manipulable por fuera de su config estática) y realiza las llamadas seguras a PRTG para evitar exponer el token de API maestro en el cliente final.
3. **Capa Frontend (Canvas Interactivo):** Renderiza los nodos de red sobre una imagen de fondo (ej. plano de oficinas, rack o diagrama lógico), permitiendo moverlos y guardando sus nuevas coordenadas en la base de datos de Django.

3. INTEGRACIÓN CON LA API DE PRTG

(<https://nova.slability.com/>)

Para obtener el inventario de sensores y sus estados en tiempo real, Django utilizará el motor de peticiones stateless de la API de PRTG.

3.1. Autenticación Segura (Server-to-Server)

Django debe inyectar las credenciales en la cabecera HTTP de cada petición o como parámetro URL cifrado:

- **Método Recomendado (Cabecera OAuth2):**
- HTTP

None

`Authorization: Bearer <mi_api_key_de_scripting>`

-

- **Método Alternativo (Query Params):**
`https://nova.slability.com/api/table.json?apitoken=<mi_api_key>` o
utilizando `username` y `passhash`.

3.2. Endpoints de Consulta Requeridos

- **Obtención del Listado Global de Sensores y Estados Actuales:**
- HTTP

None

`GET`

`https://nova.slability.com/api/table.json?content=sensors&columns=objid,sensor,status,lastvalue,device,group&count=50000`

-

- **Obtención de Metadatos de Dispositivos (Geolocalización o Grupos):**
- HTTP

None

GET

```
https://nova.slability.com/api/table.json?content=devices&columns=objid,name,location,group,probe&count=10000
```

-

3.3. Mapeo de Estados a Código de Colores Visuales

El agente debe programar un parser que convierta el parámetro `status` entregado por el JSON de PRTG al color correspondiente en el canvas interactivo:

Código de Estado en PRTG	Estado Operacional	Color en Canvas	Hexadecimal Sugerido
Up	Activo / OK	Verde	#00B050
Warning / Unusual	Advertencia / Anomalía	Amarillo / Naranja	#FFC800
Down / Down Partial	Crítico / Caído	Rojo	#DF2020
Paused	Pausado programado o manual	Gris / Azul	#9B9B9B / #4A90E2

4. DISEÑO DE LA BASE DE DATOS EN DJANGO (MODELOS ORM)

Dado que PRTG almacena internamente la configuración de mapas en su archivo binario consolidado `PRTG Configuration.dat`, no es viable ni flexible manipular directamente esas coordenadas desde ahí. Django debe persistir la información visual.

El desarrollador debe implementar los siguientes modelos en `models.py`:

Python

None

```
from django.db import models
```

```
class CustomNetworkMap(models.Model):
```

```

"""
Representa el mapa o lienzo personalizado (Canvas).
"""

name = models.CharField(max_length=255, verbose_name="Nombre
del Mapa")
background_image = models.ImageField(
    upload_to="maps/backgrounds/",
    blank=True,
    null=True,
    help_text="Imagen de fondo (Plano, rack, topología lógica)"
)
width = models.IntegerField(default=1200, help_text="Ancho del
lienzo en píxeles")
height = models.IntegerField(default=800, help_text="Alto del
lienzo en píxeles")
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

def __str__(self):
    return self.name

class NetworkMapNode(models.Model):
    """
    Representa un nodo dinámico dentro de un mapa específico,
    vinculado a un objeto de PRTG.
    """

    NODE_TYPES =

    map = models.ForeignKey(CustomNetworkMap,
on_delete=models.CASCADE, related_name="nodes")
    prtg_id = models.IntegerField(help_text="ID del objeto real
(objid) en nova.slability.com")
    node_type = models.CharField(max_length=15, choices=NODE_TYPES,
default='sensor')

```

```

        # Coordenadas de posición absoluta/relativa para el Canvas
visual
        pos_x = models.FloatField(default=100.0, help_text="Posición X
en el lienzo")
        pos_y = models.FloatField(default=100.0, help_text="Posición Y
en el lienzo")

        # Atributos estéticos opcionales
        custom_label = models.CharField(max_length=255, blank=True,
null=True, help_text="Etiqueta personalizada opcional")
        icon_style = models.CharField(max_length=50, default='server',
help_text="Estilo del icono (switch, router, db, etc.)")

class Meta:
    unique_together = ('map', 'prtg_id')

def __str__(self):
    return f"Node PRTG_ID {self.prtg_id} on {self.map.name}"

```

5. FRONTEND: LIENZO INTERACTIVO Y MANIPULACIÓN VISUAL (DRAG & DROP)

Para implementar el mapa interactivo con manipulación visual ágil, el desarrollador debe utilizar una librería de grafos robusta en JavaScript.

5.1. Librería Recomendada: Cytoscape.js (o alternativamente vis-network)

- **Razón:** `Cytoscape.js` es altamente optimizada para renderizar redes complejas, soporta arrastrar y soltar de forma nativa (`draggable: true`), responde instantáneamente a eventos táctiles y de mouse, y permite exportar fácilmente el estado del layout a formato JSON para persistencia.

5.2. Lógica del Flujo de Interacción en JavaScript:

1. **Inicialización:** Al cargar el mapa en el navegador, el frontend realiza una petición HTTP a Django `GET /api/maps/<map_id>/data/`.
2. **Renderizado de Fondo:** Se establece la `background_image` como fondo del contenedor div del mapa.

3. **Mapeo de Nodos:** Se renderizan los nodos en las coordenadas `pos_x` y `pos_y` recuperadas del JSON de Django.
4. **Guardado de Posiciones (Drag-and-Drop):** Cuando el operador arrastra un elemento y lo suelta (evento `free` o `dragend` en JavaScript), el frontend debe disparar una llamada asíncrona (Fetch/AJAX):
5. JavaScript

None

```
// Capturar evento al soltar el nodo en Cytoscape.js
cy.on('free', 'node', function(evt){
  let node = evt.target;
  let prtgId = node.id();
  let newPosition = node.position(); // Retorna { x, y }

  // Enviar coordenadas actualizadas a Django
  fetch(`/api/maps/update-node-position/`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRFToken': getCookie('csrftoken') // Token de
seguridad nativo de Django
    },
    body: JSON.stringify({
      prtg_id: prtgId,
      pos_x: newPosition.x,
      pos_y: newPosition.y
    })
  });
});
```

6.

6. SINCRONIZACIÓN Y PERFORMANCE DE ALERTA EN TIEMPO REAL

Para asegurar que las alertas se actualicen en tiempo real sin degradar el rendimiento del servidor de Django ni de nova.slability.com:

1. **Pooling en el Frontend:** El navegador ejecutará un intervalo de actualización periódica (cada 10 o 15 segundos) mediante `setInterval()` para consultar los estados dinámicos.

2. **Controlador Proxy en Django:** El navegador no interroga directamente a PRTG. En su lugar, llama a un endpoint en Django:
3. Python

None

```
# En Django Views
from django.http import JsonResponse
import requests

def get_live_statuses(request, map_id):
    # 1. Obtener todos los prtg_ids asociados a este mapa
    nodes = NetworkMapNode.objects.filter(map_id=map_id)
    prtg_ids = [node.prtg_id for node in nodes]

    # 2. Hacer llamada batch a PRTG consultando por estos IDs
    # específicos para optimizar carga
    # Utilizando la API HTTP de nova.slability.com
    prtg_url = "https://nova.slability.com/api/table.json"
    params = {
        "content": "sensors",
        "columns": "objid,status,lastvalue",
        "apitoken": "TU_API_KEY_AQUI",
        "count": 50000
    }

    try:
        response = requests.get(prtg_url, params=params, timeout=5)
        data = response.json()

        # 3. Filtrar y emparejar estados
        live_data = {}
        for sensor in data.get("sensors",):
            sensor_id = sensor.get("objid")
            if sensor_id in prtg_ids:
                live_data[sensor_id] = {
                    "status": sensor.get("status"),
                    "lastvalue": sensor.get("lastvalue")
                }
    }
```

```
        return JsonResponse({"success": True, "statuses":  
live_data})  
    except Exception as e:  
        return JsonResponse({"success": False, "error": str(e)},  
status=500)
```

- 4.
5. **Actualización Estética Dinámica:** Al recibir el JSON en el frontend, el script de Javascript debe iterar sobre los elementos del canvas y actualizar las clases CSS del nodo o las propiedades visuales del elemento de Cytoscape de inmediato, cambiando su borde o color de fondo a Rojo, Amarillo o Verde según corresponda.

7. REQUISITOS DE ENTREGABLES PARA EL DESARROLLADOR

El agente debe entregar:

- **Modelos de Django** estructurados y con sus respectivas migraciones listas para PostgreSQL o SQLite.
- **Vistas de API (Django Rest Framework o Standard Views)** optimizadas con manejo de caché básico para mitigar bloqueos o penalizaciones por exceso de peticiones a la API de PRTG.
- **Frontend interactivo responsivo** utilizando Bootstrap/Tailwind CSS y Cytoscape.js integrado en plantillas de Django.
- **Documentación** con las variables de entorno necesarias para la configuración (URL de PRTG, Token de API, etc.) para su despliegue directo en el entorno cloud.